
Path to Least Non-Determinism: Evolving Agent Skills from Natural Language to Verified Hybrid Workflows

Sai Krishna, Asit Sahoo

Author affiliations, addresses, and corresponding-author details will be added before submission.

Article type: Technical Research Article **DOI:** To be assigned by the journal **Received / Accepted / Published:** Journal supplied

Abstract

Agent skills often describe every workflow step in natural language. This is flexible, but it can make a language model repeat simple work on every run. We present Path to Least Non-Determinism (PLaND), a small evaluation-driven method for changing only the steps that can safely become code. A step may stay as an English instruction, move to a focused reference, or become a Python or Bash command. PLaND freezes the model, system prompt, agent harness, evaluation data, scorer, datasource snapshot, seed, and permissions. It accepts a hybrid skill only when accuracy stays above a fixed floor and the chosen expense measure improves. We tested the method on 20 audited OCR documents from 10 classes with a local Qwen3-14B model. The natural-language and hybrid skills both achieved 10/10 validation accuracy. The hybrid skill reduced aggregate validation tokens from 117,259 to 89,942, a reduction of 23.30%, and reduced mean latency from 11.68 to 8.98 seconds, a reduction of 23.16%. The accepted skill kept semantic classification in English and replaced document compaction with one bounded Python command. The study is small, but it shows a practical point: code should replace model reasoning only when end-to-end evidence shows that the change is safe and useful.

Keywords: agent skills; deterministic workflows; language-model agents; evaluation; token efficiency; document classification; hybrid systems

1. Introduction

Large language model (LLM) agents use prompts, tools, memory, and procedural instructions to complete multi-step tasks. Agent Skills package these instructions with optional references, scripts, and assets [1]. Skills are easy to read and change. They are also easy to overuse: a workflow may ask a model to interpret the same mechanical instruction thousands or millions of times.

Some steps need judgment. For example, deciding whether a document is a letter, memo, or report may depend on purpose and context. Other steps are stable. File validation, text normalization, counting, parsing, and schema checks can often be performed by ordinary code. When these stable steps remain only in English, the model may spend tokens and time deciding how to perform work whose behavior is already known.

The opposite approach is also risky. A developer may replace a useful semantic instruction with a brittle keyword rule. The new code can be fast and fully deterministic while making the final answer worse. The useful question is not "How much natural language can we remove?" It is:

What is the least non-deterministic version of a skill that still meets its measured quality and operating constraints?

We call this approach **Path to Least Non-Determinism (PLaND)**. PLaND treats one standard operating procedure (SOP) step as the unit of change. It measures the whole workflow before accepting any replacement. The method makes four contributions:

- 1 It gives every SOP step one of three forms: English, reference, or command.
- 2 It freezes the system prompt and experimental boundaries after the baseline.
- 3 It treats accuracy as a floor while optimizing tokens, latency, or cost.
- 4 It saves exact before-and-after skill snapshots, traces, metrics, and accept-or-reject decisions.

Our proof-of-concept uses document classification because it contains both mechanical work and semantic judgment. The result is an accepted hybrid skill: one command compacts long OCR text, while four English steps preserve the task decision.

2. Background and Related Work

2.1 Agent skills

The Agent Skills specification defines a directory with a required `SKILL.md` file and optional `scripts/`, `references/`, and `assets/` resources [1]. The format supports progressive disclosure: an agent sees small metadata first and loads detailed instructions only when the skill is needed. DeepAgents adds a runtime with tools, filesystem backends, subagents, and skill loading [2]. PLaND uses this packaging model but focuses on how the internal steps should evolve.

Recent work treats skills as a distinct layer between a model and its tools. Surveys describe skills as reusable packages of instructions, code, policies, and resources [3, 4]. Trace-conditioned methods such as SkillRevise revise skills using execution evidence and measured utility [5]. PLaND shares the use of traces, but its search target is narrower: find which individual English steps can safely move toward deterministic execution.

2.2 Prompt, workflow, and agent optimization

PLaND is related to systems that optimize LLM programs. DSPy compiles declarative modules and demonstrations against a metric [6]. TextGrad uses textual feedback to improve prompts and other textual variables [7]. AutoFlow generates and revises natural-language workflows [8]. AFlow searches over code-represented workflows with execution feedback [9]. Automated Design of Agentic Systems searches over prompts, tools, and control logic [10].

These systems show that LLM workflows can be optimized. PLaND asks a different question. It does not assume that the final workflow should be all prose or all code. It searches for a measured boundary between the two. It also freezes the system prompt after the baseline so that a result can be attributed to the SOP package rather than to a moving prompt.

2.3 Document classification data

The experiment uses QS-OCR-Small, an OCR-text version of the Tobacco-3482 document collection [11]. Tobacco-3482 contains 10 document classes. Recent auditing found important label problems: 11.7% of samples were reported as incorrectly labelled or outside the ontology, and 16.7% could reasonably carry more than one label [12]. We therefore used only non-empty records with one audited corrected label.

RVL-CDIP is a larger related benchmark with 400,000 images across 16 classes and standard 320,000/40,000/40,000 train, validation, and test splits [13]. It is a good next benchmark, but published analysis also warns about label definitions, overlap, and evaluation quality [14]. These issues matter because PLaND can only protect the accuracy that its evaluation data measures.

2.4 Inference efficiency and carbon

Reducing tokens and execution time can lower operating expense, but tokens are not a direct carbon measurement. Sprout reports that carbon-aware generation controls can reduce LLM inference emissions while maintaining output quality [15]. Other benchmarking work finds that inference energy depends on workload, hardware, batching, software, and response length [16, 17]. This means a shorter run may use less energy, but the relationship must be measured on the actual system. We therefore report tokens and time, and we do not claim a measured carbon reduction.

3. The PLaND Method

3.1 Inputs and outputs

PLaND takes five main inputs:

- 1 workflow requirements;
- 2 approved datasource files;
- 3 an evaluation CSV with input, expected output, and reference reasoning;
- 4 a model and runner; and
- 5 an accuracy floor, expense objective, and stopping limits.

The first skill, `generate-initial-version`, creates a DeepAgent with one workflow SOP. It generates the system prompt once. The second skill, `pland-evolver`, measures the baseline and proposes bounded changes to the SOP package.

3.2 Three forms of an SOP step

Each numbered SOP step has exactly one representation:

- **English instruction:** used for meaning, ambiguity, and judgment.
- **Focused reference:** used when detailed conditional guidance would make the main skill too long.
- **Command:** a Python or Bash operation used for mechanical, bounded, and testable work.

Each step is marked in the source as `english`, `reference`, or `command`. The runner saves these counts with the exact SOP text and its SHA-256 hash.

3.3 Frozen experiment boundary

After baseline measurement starts, PLaND freezes:

- the model and model digest;
- the system prompt;
- the normalized agent harness;
- the evaluation rows and expected outputs;
- the scorer;
- the datasource manifest and file hashes;
- the seed and model settings; and

- the filesystem and network permissions.

Evolution may change only the workflow SOP package: SKILL.md, its direct references, the tools or scripts directly invoked by the SOP, and approved dependencies needed by those scripts. A deterministic checker rejects a candidate when a frozen fingerprint is missing or different.

3.4 Objective and acceptance rule

For n evaluation cases, accuracy is

$$A = (1/n) * \sum [\text{indicator}(\text{predicted_i} = \text{expected_i})]. \quad (1)$$

Let the measured expense vector be

$$E = (T, L, C), \quad (2)$$

where T is aggregate model tokens, L is mean latency, and C is estimated service cost. One component is selected as the primary objective $f(E)$. PLaND accepts a candidate h over baseline b only when

$$A_h \geq \tau, f(E_h) \leq (1 - \delta) f(E_b), \text{ and } I_h = I_b, \quad (3)$$

where τ is the accuracy floor, δ is the required improvement, and I is the set of frozen fingerprints. Secondary latency, token, dependency, network, and security guardrails must also pass.

The relative saving for any positive metric m is

$$S_m = ((m_b - m_h) / m_b) * 100\%. \quad (4)$$

3.5 Evolution loop

The loop is simple:

- 1 Run the natural-language baseline from isolated state.
- 2 Save every output, trace, error, token count, latency, and frozen fingerprint.
- 3 Find a repeated mechanical action or a measured failure.
- 4 Propose one bounded SOP-package change.
- 5 Run unit checks for new code.
- 6 Run development evaluations under the same frozen conditions.
- 7 Reject a candidate below the accuracy floor or without expense improvement.
- 8 Run validation only for an eligible candidate.
- 9 Save the direct natural-language-versus-hybrid comparison.
- 10 Stop on success, after 10 attempts, or at an earlier cost or time limit.

The limit is 10 candidate attempts, not a target. If candidate 1 succeeds, nine extra experiments would add expense without answering the research question.

4. Experimental Method

4.1 Dataset selection

The source collection contains 3,482 OCR files across advertisement, email, form, letter, memo, news, note, report, resume, and scientific classes. We matched filenames to the independent Tobacco-3482 audit. Empty text, unmatched files, unknown labels, and multi-label audit records were excluded. This left 2,724 eligible documents.

Using seed 20260902, we sorted and shuffled each class by stable identifier. We selected two documents per class before any model prediction. The first 10 formed the development split and the second 10 formed the validation split. Expected outputs and reference reasoning were hidden from the runtime agent.

4.2 Runtime and baseline

Ollama 0.33.0 served local qwen3:14b on an Apple Silicon computer with 48 GB unified memory. The model digest was bdbd181c33f2ed1b31c972991882db3cf4d192569092138a7d29e973cd9debe8. Temperature was 0, seed was 42, and model reasoning mode was disabled. The runtime used Python 3.14.7, DeepAgents 0.7.12, and langchain-ollama 1.1.0. No paid model API was used.

The natural-language baseline had five English steps. It read the approved OCR text and asked the model to identify purpose, structure, class, and output shape. The system prompt and harness were then frozen.

4.3 Hybrid candidate

The accepted candidate changed one step. Instead of returning every long OCR file unchanged, a Python script performed bounded compaction. Documents of 3,200 characters or fewer were preserved after newline normalization. For a longer document, the script retained a 2,200-character head, a 900-character tail, an omission marker, and simple structural metadata. It did not predict a class or call another model.

The hybrid SOP therefore contained four English steps and one command step. The English steps still decided the document class. This separation is important: the code reduced repeated text handling without replacing semantic judgment.

4.4 Metrics

We measured exact-label accuracy, correct cases, input tokens, output tokens, total tokens, total latency, mean latency, nearest-rank p95 latency, errors, and estimated model-service cost. The primary objective was total tokens. The accuracy floor was 0.90 and the minimum objective improvement was 5%.

Each split contained only 10 cases. A 10/10 result has a two-sided 95% Wilson score interval of approximately 0.72 to 1.00. This wide interval is a reminder that the result describes this execution and is not a population estimate.

5. Results

5.1 Development result

Table 1 shows the development result. Both versions classified all 10 cases correctly. The hybrid version used 20,522 fewer tokens and had lower mean and p95 latency.

Table 1. Development comparison

Measure	Natural-language SOP	Hybrid SOP	Change
Accuracy	10/10 (100%)	10/10 (100%)	0 percentage points
Input tokens	109,825	89,442	-20,383
Output tokens	1,627	1,488	-139
Total tokens	111,452	90,930	-18.41%
Mean latency	10.43 s	8.87 s	-15.00%
p95 latency	24.78 s	10.56 s	-57.40%
Model-service cost	USD 0	USD 0	USD 0

The deterministic assessor marked the candidate eligible for validation. All frozen fingerprints matched.

5.2 Validation result

Table 2 reports the frozen validation result. Accuracy remained 10/10. The hybrid SOP used 27,317 fewer tokens and reduced mean latency by 2.71 seconds.

Table 2. Validation comparison

Measure	Natural-language SOP	Hybrid SOP	Change
Accuracy	10/10 (100%)	10/10 (100%)	0 percentage points
Input tokens	115,645	88,500	-27,145
Output tokens	1,614	1,442	-172
Total tokens	117,259	89,942	-23.30%
Mean latency	11.68 s	8.98 s	-23.16%
p95 latency	24.74 s	10.94 s	-55.78%
Model-service cost	USD 0	USD 0	USD 0

Candidate 1 passed the accuracy floor, the 5% token-improvement rule, the latency guardrail, the hybrid-SOP requirement, and every frozen-invariant check. It was accepted, so the search stopped after one of at most 10 attempts.

5.3 Scale illustration

The measured validation saving was 27,317 tokens across 10 documents, or about 2,732 tokens per document. If the same difference held for N comparable documents, the illustrative token saving would be

$$T_{\text{save}}(N) = N * (117,259 - 89,942) / 10. (5)$$

For one million comparable documents, this equals about 2.73 billion tokens. The mean-latency difference was 2.71 seconds per document. Across one million documents, this is about 2.71 million seconds, or 31 days of aggregate compute time. Parallel serving changes calendar time, so this is not a claim that a production deployment would finish 31 days earlier.

6. Discussion

6.1 What the result means

The result supports a narrow claim. A deterministic compaction step reduced measured token use and latency on this small document set without changing any label. The model still made the semantic decision. PLaND did not make the whole workflow deterministic; it removed one source of unnecessary model input.

This outcome also explains the phrase "least non-determinism." More code is not automatically better. In an earlier pilot, deterministic cue counts reduced tokens but dropped development accuracy to 70%. PLaND rejected that candidate. The accepted candidate was less ambitious and more useful: compact the text, then let the model interpret it.

6.2 Why the frozen prompt matters

If the system prompt changes with every candidate, it is difficult to know why the score changed. The final experiment freezes the prompt and normalized harness. Only SOP-package changes are allowed. This makes the comparison easier to audit and closer to a controlled software experiment.

6.3 Enterprise scale

A 23.30% reduction may look small in a 10-document validation set. At enterprise volume, repeated savings can be material. Many organizations process millions of documents, messages, or agent steps. Even a moderate per-run improvement can accumulate into billions of avoided tokens and large amounts of compute time.

The scale calculation is not a forecast. Real savings depend on document mix, batching, caching, concurrency, model serving, hardware, and price. A larger study must measure these conditions directly. The simple point is that a stable mechanical step is paid for on every run when it remains model-mediated.

6.4 Carbon and energy, stated carefully

We did not measure electrical power, energy, or grid carbon intensity. We therefore make no numerical carbon claim. A future experiment can measure average power P in watts over runtime t in seconds:

$$E_{\text{kWh}} = (P * t) / (3.6 * 10^6). (6)$$

With grid carbon intensity g in grams of CO₂-equivalent per kWh, operational emissions are

$$\text{CO2e} = \text{E_kWh} * \text{g}. (7)$$

This simple calculation should be applied to measured power and time for both skills. Token and latency reductions are useful signals, but they are not substitutes for energy measurement. Hardware manufacture, idle power, cooling, and shared infrastructure would also need to be considered for a fuller carbon account.

7. Limitations and Threats to Validity

This study has important limits.

- 1 It uses only 20 documents, with one case per class in each split.
- 2 It has no third held-out split; validation was used for candidate acceptance.
- 3 Each agent and split was run once with one model and one seed.
- 4 The 10/10 accuracy estimate has a wide Wilson interval.
- 5 The dataset contains OCR noise, historical tobacco-industry content, label ambiguity, and possible privacy concerns.
- 6 The study uses OCR text, not raw images or PDFs.
- 7 Token counts came from the model integration and are aggregate usage across agent turns; they are not unique source tokens.
- 8 Latency depends on local model state and system contention.
- 9 No energy or carbon measurement was collected.
- 10 The compaction thresholds may not transfer to longer or more diverse documents.
- 11 The accepted result may not reproduce across other models or frameworks.

These limits make the work a proof-of-concept, not a benchmark result. The next study should use a larger stratified RVL-CDIP subset, a true held-out set, repeated seeds, more local models, confidence intervals, and direct energy measurement.

8. Security, Privacy, and Reproducibility

Raw OCR and case traces are kept outside version control because source documents may contain sensitive information. The repository stores selection code, stable identifiers, hashes, safe aggregate metrics, exact SOP snapshots, and deterministic accept-or-reject records. The hybrid script validates each requested path against an approved manifest, uses no shell, has bounded input and output, and calls no remote service.

The experiment code, skill packages, tests, comparison JSON, and decision JSON are versioned. A reader can inspect the frozen hashes for the prompt, harness, datasource snapshot, evaluation file, and scorer. This does not replace

an independent replication, but it makes accidental boundary changes visible.

9. Conclusion

PLaND is a simple method for evolving agent skills toward verified hybrid workflows. It keeps semantic judgment in natural language and moves only stable, bounded work into code. Accuracy is a floor, not the only objective, and the system prompt and experiment boundary stay frozen after baseline measurement.

In the reported proof-of-concept, one Python compaction step preserved 10/10 validation accuracy, reduced aggregate tokens by 23.30%, and reduced mean latency by 23.16%. The sample is too small for broad claims, but the mechanism is clear and reproducible. At larger operating volumes, safe per-run savings can compound. Future work should test this idea on larger audited datasets and measure energy directly.

Acknowledgments

The authors thank contributors who reviewed the workflow and dataset options. Specific acknowledgments and funding information will be added with permission.

Conflict of Interest

The authors will provide the final journal-required conflict-of-interest statement before submission.

Data and Code Availability

The implementation, skill definitions, selection code, and aggregate evidence are maintained in the project repository. Raw source documents and traces are not redistributed. Repository URL and archival identifier will be added before submission.

References

- [1] Agent Skills. "Agent Skills Specification." <https://agentskills.io/specification>
- [2] LangChain. "DeepAgents: Skills." <https://docs.langchain.com/oss/python/deepagents/skills>
- [3] R. Xu and Y. Yan. "Agent Skills for Large Language Models: Architecture, Acquisition, Security, and the Path Forward." arXiv:2602.12430, 2026. <https://arxiv.org/abs/2602.12430>
- [4] Y. Jiang et al. "SoK: Agentic Skills - Beyond Tool Use in LLM Agents." arXiv:2602.20867, 2026. <https://arxiv.org/abs/2602.20867>
- [5] Y. Liu et al. "SkillRevise: Improving LLM-Authoring Agent Skills via Trace-Conditioned Skill Revision." arXiv:2606.01139, 2026. <https://arxiv.org/abs/2606.01139>
- [6] O. Khattab et al. "DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines." arXiv:2310.03714, 2023. <https://arxiv.org/abs/2310.03714>

-
- [7] M. Yuksekgonul et al. "TextGrad: Automatic Differentiation via Text." arXiv:2406.07496, 2024. <https://arxiv.org/abs/2406.07496>
- [8] Z. Li et al. "AutoFlow: Automated Workflow Generation for Large Language Model Agents." arXiv:2407.12821, 2024. <https://arxiv.org/abs/2407.12821>
- [9] J. Zhang et al. "AFlow: Automating Agentic Workflow Generation." arXiv:2410.10762, 2024. <https://arxiv.org/abs/2410.10762>
- [10] S. Hu, C. Lu, and J. Clune. "Automated Design of Agentic Systems." Advances in Neural Information Processing Systems, 2024. <https://arxiv.org/abs/2408.08435>
- [11] QuickSign. "Quicksign OCRized Text Dataset (QS-OCR)." <https://github.com/QuickSign/ocrized-text-dataset>
- [12] G. Lim, S. Larson, and K. Leach. "Label Errors in the Tobacco3482 Dataset." arXiv:2412.13140, 2024. <https://arxiv.org/abs/2412.13140>
- [13] A. W. Harley, A. Ufkes, and K. G. Derpanis. "Evaluation of Deep Convolutional Nets for Document Image Classification and Retrieval." ICDAR, 2015. <https://arxiv.org/abs/1502.07058>
- [14] S. Larson, G. Lim, and K. Leach. "On Evaluation of Document Classification using RVL-CDIP." arXiv:2306.12550, 2023. <https://arxiv.org/abs/2306.12550>
- [15] B. Li, Y. Jiang, V. Gadepally, and D. Tiwari. "Sprout: Green Generative AI with Carbon-Efficient LLM Inference." EMNLP, 2024, pp. 21799-21813. <https://aclanthology.org/2024.emnlp-main.1215/>
- [16] J. Fernandez et al. "Energy Considerations of Large Language Model Inference and Efficiency Optimizations." ACL, 2025. <https://aclanthology.org/2025.acl-long.1563/>
- [17] S. Poddar et al. "Towards Sustainable NLP: Insights from Benchmarking Inference Energy in Large Language Models." NAACL, 2025. <https://aclanthology.org/2025.naacl-long.632/>